

Weather Forecasting Using Feature Engineering and Comparative Machine Learning and Neural Network Models

Anjali Yadav, Veeru

*5th Semester, Electronics and Communication Engineering (ECE),
Bharati Vidyapeeth College of Engineering, Pune, India*

Abstract

Accurate weather forecasting is important for applications such as agriculture, transportation, energy management, environmental monitoring, and disaster preparedness. Weather variables exhibit temporal dependencies, seasonal patterns, and nonlinear relationships that make short-term forecasting a challenging machine learning problem. This study develops a supervised learning framework for forecasting the next-day mean temperature using historical weather observations and engineered temporal features.

The proposed framework first preprocesses weather observations by converting sub-daily measurements to daily averages when required and handling small missing-value gaps through time-based interpolation. Feature engineering is then performed using calendar-based cyclical features, lag variables, rolling statistics, and first-order difference features. A chronological train-test split is used to preserve the temporal structure of the data and prevent future information from entering the training process.

Six machine learning models are evaluated: Linear Regression, Ridge Regression, Lasso Regression, Random Forest, Gradient Boosting, and a feed-forward Neural Network based on a Multi-Layer Perceptron (MLP). Persistence and monthly climatology are additionally used as baseline forecasting methods. Hyperparameter selection for the machine learning models is performed using four-fold TimeSeriesSplit cross-validation and GridSearchCV. Model performance is evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and coefficient of determination (R^2).

In addition to model comparison, a feature-engineering ablation experiment is conducted to determine whether engineered temporal features improve forecasting performance compared with raw weather variables alone. The framework also produces actual-versus-predicted plots, residual diagnostics, model comparison plots, and feature-importance analysis.

Keywords: Weather forecasting, Machine learning, Feature engineering, Time-series forecasting, Regression, Random Forest, Gradient Boosting, Neural Network, Multi-Layer Perceptron

1. Introduction

Weather forecasting is a significant application of data-driven modelling because atmospheric variables vary continuously over time and are influenced by seasonal, temporal, and environmental factors. Traditional forecasting methods rely heavily on physical models and statistical approaches, whereas machine learning methods can learn relationships directly from historical observations.

Temperature forecasting is particularly suitable for supervised machine learning because historical temperature, humidity, pressure, wind, rainfall, and cloud-related observations contain useful information about future conditions. However, directly applying machine learning algorithms to raw weather measurements may not adequately represent temporal dependencies.

Feature engineering provides a method of transforming raw observations into more informative predictors. Lagged variables represent previous atmospheric conditions, while rolling statistics capture recent trends and variability. Calendar-based cyclical transformations can represent seasonal behaviour without introducing artificial discontinuities between the end and beginning of a year.

The objective of this research is to develop and evaluate a machine learning framework for next-day mean temperature forecasting. The study compares traditional regression models, ensemble tree-based models, and a neural network model using the same engineered feature set.

The main objectives of the study are:

1. To preprocess historical weather observations for supervised forecasting.
2. To perform temporal feature engineering using calendar, lag, rolling, and difference features.
3. To compare linear regression, regularized regression, ensemble tree-based models, and a neural network.
4. To evaluate the models using MAE, RMSE, and R^2 .
5. To investigate the contribution of feature engineering using an ablation experiment.
6. To analyse forecasting errors using prediction plots and residual diagnostics.

2. Related Work

Machine learning has increasingly been applied to weather forecasting because of its ability to model nonlinear relationships among meteorological variables. Linear regression provides a simple baseline and is useful for understanding approximately linear relationships between predictors and the forecast target [1, 2].

Regularized regression methods such as Ridge and Lasso introduce regularization to improve model stability when predictors are correlated or when a large number of engineered features are used. Ridge regression reduces the magnitude of model coefficients, while Lasso can also perform implicit feature selection [5, 6].

Tree-based ensemble methods provide an alternative approach for modelling nonlinear relationships. Random Forest combines predictions from multiple decision trees [3], while Gradient Boosting constructs an ensemble sequentially by reducing prediction errors from previous learners [4].

Neural networks provide another class of nonlinear forecasting models. A Multi-Layer Perceptron (MLP) can learn nonlinear mappings between engineered weather features and the target temperature [7]. Because weather data contains variables with different numerical scales, standardization is particularly important for neural-network and linear-model optimization.

The present study focuses on a controlled comparison in which the same forecasting dataset and engineered feature representation are supplied to multiple model families. This makes it possible to compare the behaviour of linear, regularized, ensemble, and neural-network approaches under the same experimental conditions.

3. Dataset and Data Preprocessing

3.1. Dataset Structure

The forecasting framework accepts a CSV dataset containing a date column and a target temperature column. Additional numeric meteorological variables can also be supplied as predictors.

The framework is designed to work with variables such as:

- Temperature
- Humidity
- Atmospheric pressure
- Wind
- Cloud-related measurements
- Rainfall

The implementation also contains a synthetic-weather generator for testing the forecasting pipeline when no external CSV dataset is provided. The synthetic data contains seasonal behaviour, autocorrelated atmospheric variation, and relationships between meteorological variables. The primary experimental target is the next-day mean temperature.

3.2. Data Cleaning

The input date column is converted into a datetime representation and the observations are sorted chronologically. Only numeric weather variables and the date variable are retained for modelling.

Columns with no variation are removed because they cannot provide predictive information.

If the input dataset contains sub-daily observations, the implementation resamples the observations into one row per day using daily averages. Small missing-value gaps are subsequently handled using time-based interpolation.

This preprocessing procedure ensures that the resulting dataset has a consistent daily temporal resolution.

4. Problem Formulation

Let the weather observations at time t be represented by

$$X_t = [x_{1,t}, x_{2,t}, \dots, x_{p,t}] \quad (1)$$

where p represents the number of available weather variables.

The objective is to predict the mean temperature at a future time $t + h$, where h is the forecasting horizon.

For next-day forecasting,

$$h = 1 \quad (2)$$

and the target is defined as

$$Y_t = T_{t+1} \quad (3)$$

where T_{t+1} is the mean temperature of the following day.

After feature engineering, the model input becomes

$$X'_t = [X_t, C_t, L_t, R_t, D_t] \quad (4)$$

where:

- C_t represents cyclical calendar features,
- L_t represents lag features,
- R_t represents rolling statistics,
- D_t represents difference features.

The forecasting problem can therefore be expressed as

$$\hat{Y}_{t+h} = f(X'_t) \quad (5)$$

where f represents the selected machine learning or neural-network model.

5. Feature Engineering

Feature engineering is one of the main components of the proposed forecasting framework. The implementation constructs features using only information available at the current time t , thereby avoiding direct target leakage.

5.1. Raw Weather Features

The current-day numerical weather variables are retained as predictors. These may include temperature, humidity, pressure, wind, rainfall, and cloud measurements depending on the input dataset.

5.2. Cyclical Calendar Features

Calendar variables are transformed using sine and cosine functions to represent their cyclic nature.

For day of year:

$$D_{\sin} = \sin\left(\frac{2\pi \cdot \text{DOY}}{365.25}\right) \quad (6)$$

$$D_{\cos} = \cos\left(\frac{2\pi \cdot \text{DOY}}{365.25}\right) \quad (7)$$

Similarly, month information is represented using

$$M_{\sin} = \sin\left(\frac{2\pi \cdot \text{Month}}{12}\right) \quad (8)$$

and

$$M_{\cos} = \cos\left(\frac{2\pi \cdot \text{Month}}{12}\right). \quad (9)$$

These transformations allow December and January to be represented as neighbouring points in the feature space rather than distant numerical values.

5.3. Lag Features

Lag features represent historical observations. The implementation uses lags of 1, 2, 3, and 7 days.

For a variable x ,

$$L_{t,k} = x_{t-k} \quad (10)$$

where

$$k \in \{1, 2, 3, 7\}.$$

These features provide the models with information about recent atmospheric conditions.

5.4. Rolling Mean Features

Rolling means are calculated over windows of 3, 7, and 14 days.

$$R_{t,w} = \frac{1}{w} \sum_{i=0}^{w-1} x_{t-i} \quad (11)$$

where

$$w \in \{3, 7, 14\}.$$

Rolling averages help represent short-term and medium-term trends.

5.5. Rolling Standard Deviation

A seven-day rolling standard deviation is also calculated:

$$S_t = \text{std}(x_{t-6}, \dots, x_t) \quad (12)$$

This feature represents recent variability in the weather variable.

5.6. Difference Features

First-order differences are calculated as

$$\Delta x_t = x_t - x_{t-1}. \quad (13)$$

For atmospheric pressure, the implementation additionally calculates a three-day difference:

$$\Delta P_{t,3} = P_t - P_{t-3}. \quad (14)$$

These features allow the models to represent whether weather variables are increasing or decreasing rather than considering only their absolute values.

6. Proposed Methodology

The complete forecasting framework consists of the following stages:

1. Data loading
2. Data preprocessing
3. Feature engineering
4. Target construction
5. Chronological train-test splitting
6. Model training and hyperparameter tuning
7. Forecast generation
8. Performance evaluation
9. Feature-engineering ablation
10. Visualization and residual analysis

The chronological structure is preserved throughout the experiment. The implementation does not randomly shuffle the observations before creating the train-test partitions.

7. Pseudocode

The pseudocode below summarizes the complete forecasting procedure used in the implementation.

Algorithm 1: Weather Forecasting Using Feature Engineering

Require: Weather dataset D , forecasting horizon h
Ensure: Predictions $\hat{Y}$ and evaluation metrics

- 1: Load weather dataset D
- 2: Convert date column to datetime format
- 3: Sort observations chronologically
- 4: Retain date and numerical weather variables
- 5: Remove constant-valued variables
- 6: **if** data frequency is less than one day **then**
- 7: Resample observations into daily averages
- 8: **end if**
- 9: Interpolate small missing-value gaps using time-based interpolation
- 10: Remove remaining incomplete observations
- 11: **for** each weather variable x **do**
- 12: Add current value x_t
- 13: Generate lag features for $k \in \{1, 2, 3, 7\}$
- 14: Generate rolling means for $w \in \{3, 7, 14\}$
- 15: Generate seven-day rolling standard deviation
- 16: Generate first-order difference Δx_t
- 17: **end for**
- 18: Generate day-of-year sine and cosine features
- 19: Generate month sine and cosine features
- 20: **if** pressure variable exists **then**
- 21: Generate three-day pressure difference
- 22: **end if**
- 23: Define target as $Y_t = T_{t+h}$
- 24: Remove rows containing invalid feature or target values
- 25: Split data chronologically into training and testing sets
- 26: Use approximately 80% observations for training
- 27: Use approximately 20% observations for testing
- 28: Define Persistence and Climatology baseline models
- 29: Define Linear Regression
- 30: Define Ridge Regression
- 31: Define Lasso Regression
- 32: Define Random Forest
- 33: Define Gradient Boosting
- 34: Define MLP Neural Network
- 35: Apply standardization to models requiring feature scaling
- 36: Create four-fold TimeSeriesSplit cross-validation
- 37: Perform GridSearchCV for models with hyperparameter grids
- 38: **for** each model **do**
- 39: Train model using training data

```

40:   Generate predictions for test data
41:   Calculate MAE
42:   Calculate RMSE
43:   Calculate  $R^2$ 
44:   Store predictions and model parameters
45: end for
46: Compare model performance
47: Generate actual-versus-predicted plots
48: Generate residual diagnostics
49: Generate feature importance for Random Forest
50: Compare raw features against engineered features
51: Save model comparison and feature-ablation results
    return Predictions, metrics, plots, and saved results

```

8. Forecasting Models

8.1. Linear Regression

Linear Regression is used as a basic regression model. The predicted temperature is represented as

$$\hat{y} = \beta_0 + \sum_{j=1}^p \beta_j x_j. \quad (15)$$

It provides a simple reference for evaluating more complex nonlinear models.

8.2. Ridge Regression

Ridge Regression adds an L_2 regularization term:

$$\min_{\beta} \left[\sum_{i=1}^N (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^p \beta_j^2 \right]. \quad (16)$$

The implemented hyperparameter search evaluates

$$\alpha \in \{0.1, 1, 10, 100\}.$$

8.3. Lasso Regression

Lasso Regression uses an L_1 regularization penalty:

$$\min_{\beta} \left[\sum_{i=1}^N (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^p |\beta_j| \right]. \quad (17)$$

The tested values of α are

$$\{0.001, 0.01, 0.1\}.$$

8.4. Random Forest

Random Forest is an ensemble of decision trees. Each tree produces an individual prediction and the final regression output is obtained by aggregating the tree predictions [3].

The implemented hyperparameter search considers:

- Number of estimators: 200
- Maximum depth: 8 or 16
- Minimum samples per leaf: 2 or 5

8.5. Gradient Boosting

Gradient Boosting constructs an ensemble of weak learners sequentially, where each new learner attempts to reduce the errors made by the existing ensemble [4].

The implemented search considers:

- Number of estimators: 200 or 400
- Learning rate: 0.05 or 0.1
- Maximum depth: 3

8.6. Neural Network

A feed-forward Multi-Layer Perceptron (MLP) is used as the primary neural network model. The tested hidden-layer configurations are:

(16), (32), (32, 16).

The implementation also evaluates the regularization parameter

$$\alpha \in \{0.1, 1, 10\}.$$

The initial learning rate is fixed at

$$10^{-3}$$

and the maximum number of iterations is set to 2000.

The MLP is placed inside a pipeline containing a StandardScaler so that feature scaling is performed consistently during model training and validation.

9. Baseline Forecasting Methods

Two simple forecasting baselines are included to provide reference points for the machine learning models.

9.1. Persistence Baseline

The persistence model assumes that tomorrow's temperature will be equal to today's temperature:

$$\hat{T}_{t+1} = T_t. \tag{18}$$

9.2. Monthly Climatology

The climatology baseline predicts temperature using the average target temperature for the corresponding month in the training data.

For month m :

$$\hat{T}_m = \frac{1}{N_m} \sum_{i \in m} T_i. \quad (19)$$

10. Training and Validation Strategy

10.1. Chronological Train-Test Split

The dataset is divided chronologically into training and testing partitions. The default test size is 20%, resulting in approximately 80% training data and 20% testing data.

Let N represent the number of observations. The splitting point is

$$N_{\text{train}} = \lfloor 0.8N \rfloor. \quad (20)$$

The first part of the dataset is used for training, while the final part is reserved for testing.

This strategy is important for time-series forecasting because random shuffling could allow information from later observations to influence the training process.

10.2. Time-Series Cross-Validation

Four-fold TimeSeriesSplit cross-validation is used for model selection. Unlike conventional random cross-validation, TimeSeriesSplit maintains the temporal order of observations.

10.3. Hyperparameter Optimization

GridSearchCV is used for models with predefined hyperparameter grids. The optimization objective is negative mean absolute error, corresponding to minimizing MAE.

The best estimator identified during cross-validation is subsequently fitted and evaluated on the held-out test set.

11. Evaluation Metrics

Three metrics are used to evaluate forecasting performance.

11.1. Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|. \quad (21)$$

Lower MAE indicates smaller average absolute prediction errors.

11.2. Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}. \quad (22)$$

RMSE gives greater influence to larger prediction errors.

11.3. Coefficient of Determination

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}. \quad (23)$$

Higher R^2 indicates that a larger proportion of the variation in the target is explained by the model.

12. Experimental Results

12.1. Model Comparison

The forecasting implementation evaluates the baseline methods and the machine-learning models using MAE, RMSE, and R^2 . The uploaded experimental figures are included below. The exact numerical values for the tables should be taken from the corresponding `model_comparison.csv` output so that the reported values remain consistent with the executed experiment.

Table 1: Comparison of forecasting models.

Model	MAE	RMSE	R^2
Persistence	—	—	—
Monthly Climatology	—	—	—
Linear Regression	—	—	—
Ridge Regression	—	—	—
Lasso Regression	—	—	—
Random Forest	—	—	—
Gradient Boosting	—	—	—
Neural Network (MLP)	—	—	—

12.2. Experimental Result Set 1

Figure 1 presents the RMSE and MAE comparison for the first uploaded experimental result set.

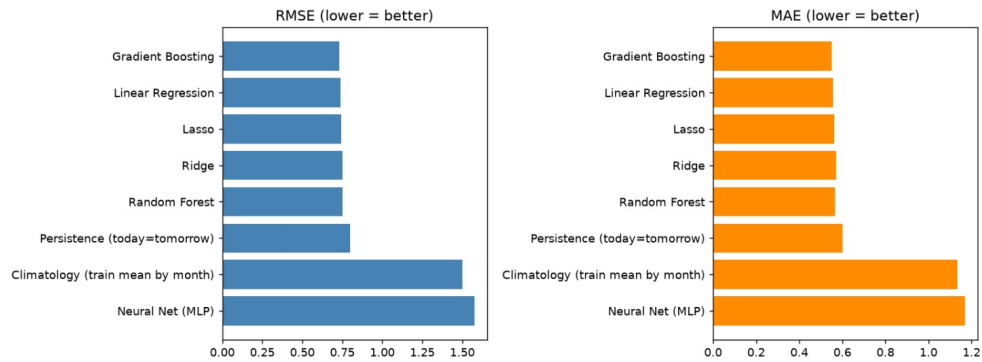


Figure 1: Model comparison using RMSE and MAE for experimental result set 1.

12.3. Forecast Versus Actual Temperature

The time-series forecast plot compares the actual temperature with the predictions generated by Linear Regression, Random Forest, and the Neural Network (MLP).

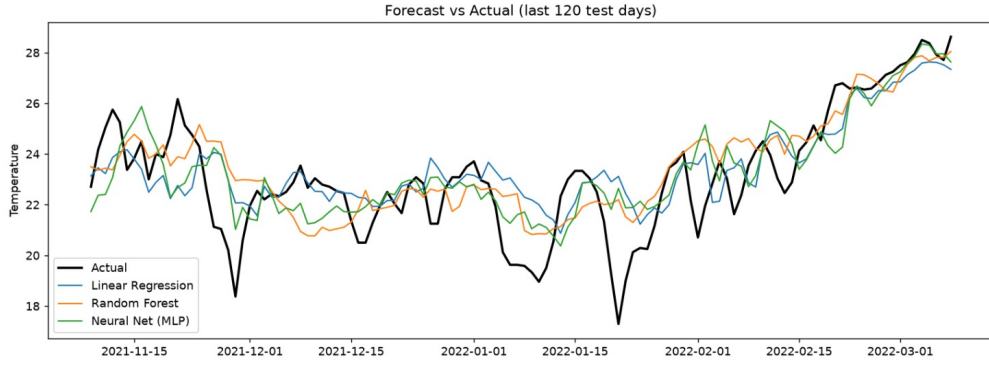


Figure 2: Forecast versus actual temperature for the final portion of the test period in experimental result set 1.

12.4. Error and Residual Analysis

Residual analysis is performed using the selected machine-learning model. The residual is defined as

$$e_i = y_i - \hat{y}_i. \quad (24)$$

The uploaded diagnostic figure contains both the predicted-versus-actual scatter plot and the residual distribution.

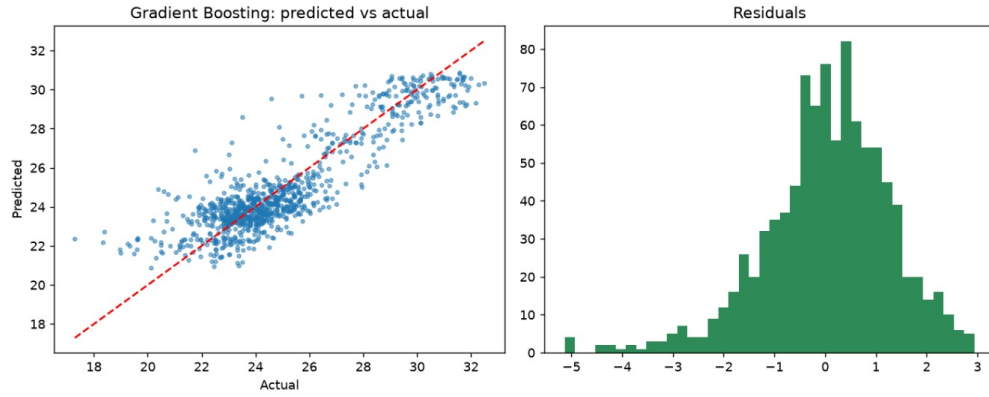


Figure 3: Predicted-versus-actual relationship and residual distribution for the selected model in experimental result set 1.

12.5. Feature Importance Analysis

Random Forest feature importance is used to examine the contribution of the engineered predictors. The implementation displays the top 15 features.

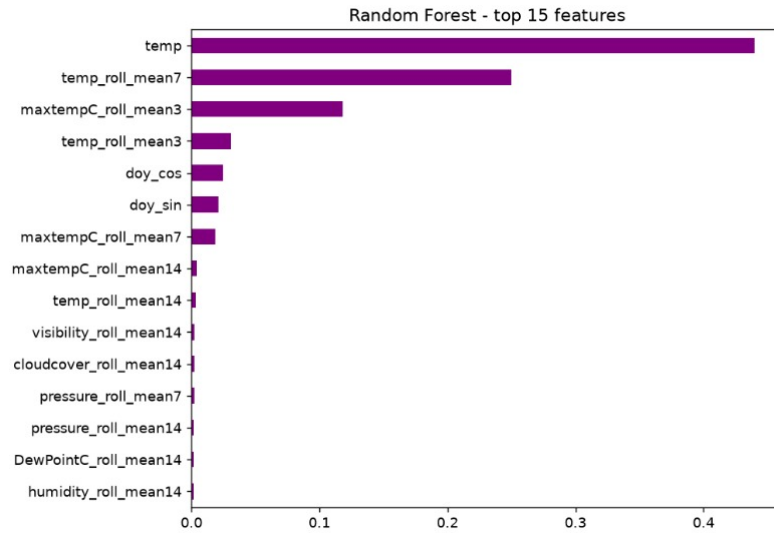


Figure 4: Top 15 Random Forest features for experimental result set 1.

12.6. Experimental Result Set 2

A second set of uploaded experimental figures is also included to document the additional run of the forecasting workflow. The figures are kept separate from the first result set rather than combining values from the two runs.

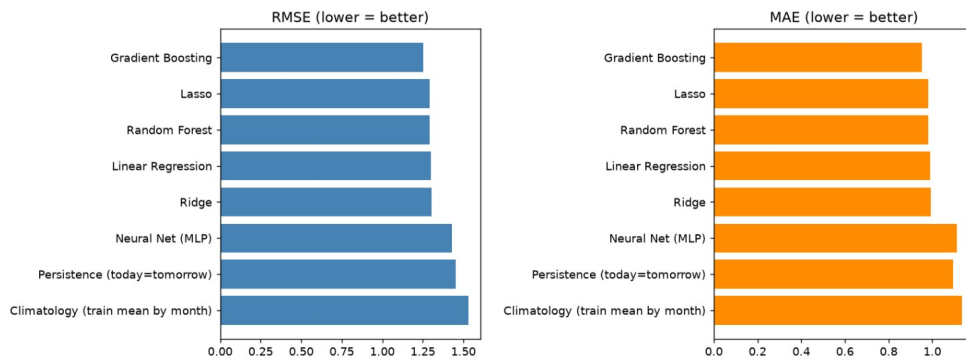


Figure 5: Model comparison using RMSE and MAE for experimental result set 2.

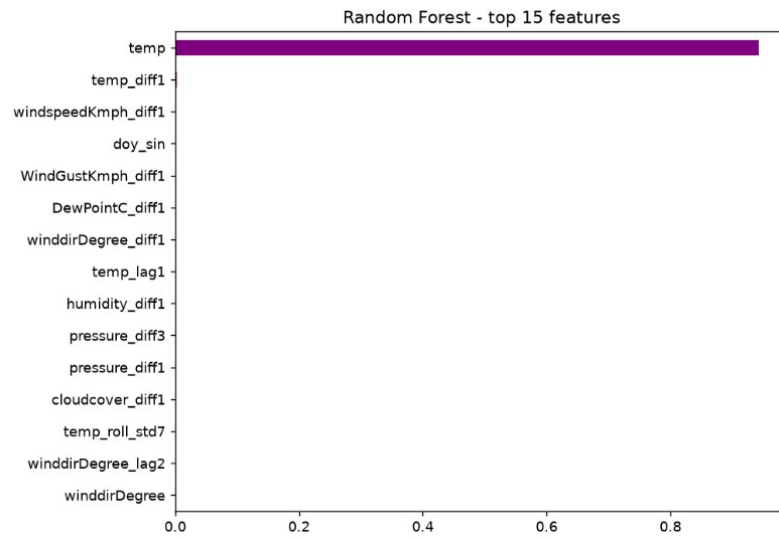


Figure 6: Top 15 Random Forest features for experimental result set 2.

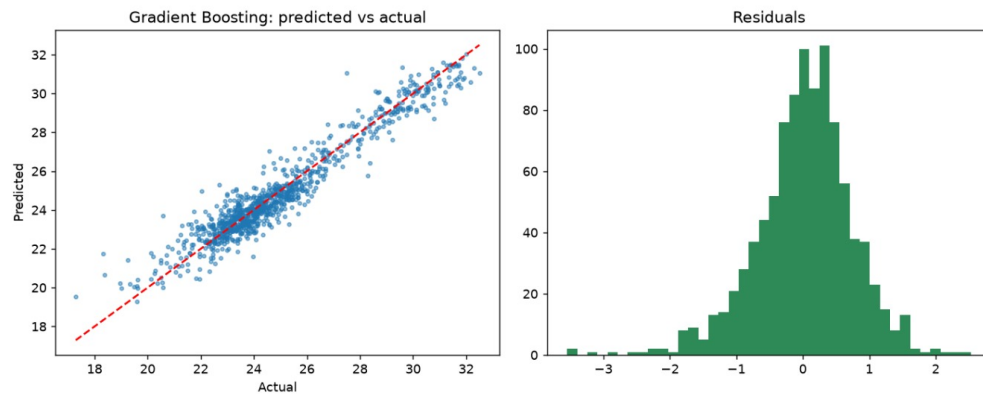


Figure 7: Predicted-versus-actual relationship and residual distribution for experimental result set 2.

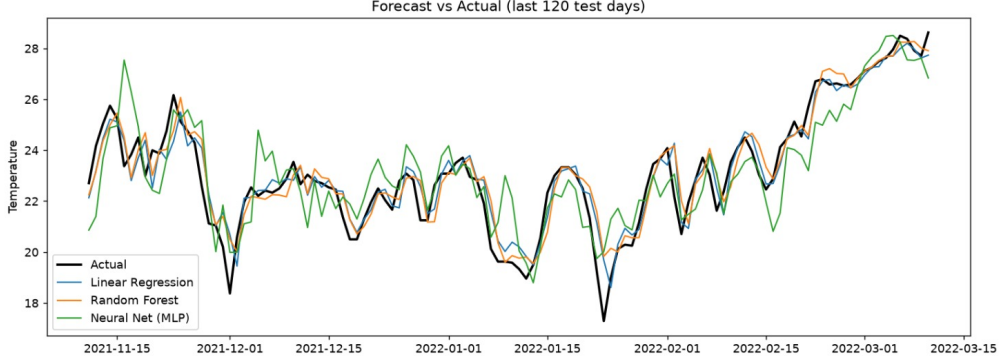


Figure 8: Forecast versus actual temperature for the final portion of the test period in experimental result set 2.

12.7. Feature Engineering Ablation Study

To evaluate the contribution of feature engineering, the implementation performs an ablation experiment using Ridge Regression.

Two feature configurations are compared:

1. Raw weather features only.
2. Raw weather features combined with engineered temporal features.

Table 2: Feature-engineering ablation using Ridge Regression.

Feature Set	No. Features	MAE	RMSE	R^2
Raw features only	—	—	—	—
Engineered features	—	—	—	—

13. Discussion

The proposed framework provides a controlled comparison between different model families for next-day temperature forecasting.

Linear Regression provides a simple reference model, while Ridge and Lasso introduce regularization that can be useful when the engineered feature space contains correlated predictors. Random Forest and Gradient Boosting provide nonlinear tree-based alternatives, whereas the MLP represents a neural-network approach.

The temporal feature-engineering process is important because the forecasting task depends not only on current weather observations but also on recent historical behaviour. Lagged features provide direct information about previous observations, while rolling statistics represent recent trends and variability. Cyclical calendar variables additionally represent seasonal structure.

The feature-ablation experiment provides a separate assessment of whether these transformations improve forecasting compared with using raw variables alone.

The final interpretation of model performance should be based on the numerical results obtained from the actual dataset rather than assumed performance differences.

14. Optional LSTM Extension

The Python implementation also contains an optional Long Short-Term Memory (LSTM) model. The LSTM component is not included in the primary model comparison because it is explicitly controlled through an optional `-lstm` command-line argument.

When enabled, the implementation uses a sequence length of 14 observations and an LSTM architecture containing two layers with 64 hidden units. The output layer produces a single temperature prediction.

The optional LSTM implementation requires PyTorch. Therefore, it can be considered an extension to the main experimental framework rather than a mandatory component of the study [8].

15. Limitations

Several limitations should be considered when interpreting the results.

First, the primary implementation forecasts the next-day mean temperature. Although the framework accepts additional meteorological variables as predictors, the target construction is specifically based on the temperature variable.

Second, the quality of the forecasting results depends on the quality, length, and geographical representativeness of the input weather dataset.

Third, the feature-engineering windows are fixed to 3, 7, and 14 days. Other temporal windows may provide different results for different locations or datasets.

Fourth, the model comparison focuses on tabular machine learning and an MLP as the main neural-network architecture. The optional LSTM provides a possible extension for explicit sequence modelling.

Finally, numerical model performance should be interpreted together with the characteristics of the dataset and forecasting horizon.

16. Reproducibility

The forecasting implementation is organized as a Python pipeline. The program accepts command-line arguments for the CSV file, date column, target column, forecasting horizon, test size, output directory, optional LSTM execution, columns to drop, and date format.

Example execution for next-day temperature forecasting is:

```
python weather_forecast.py --csv data.csv --date-col date --target temp --horizon 1
```

The program generates the following outputs:

- `model_comparison.csv`
- `feature_ablation.csv`
- `1_model_comparison.png`
- `2_forecast_vs_actual.png`
- `3_best_model_diagnostics.png`
- `4_feature_importance.png`

The implementation also uses a fixed random state of 42 for reproducibility.

17. Conclusion

This study presents a machine learning framework for next-day mean temperature forecasting using temporal feature engineering and comparative model evaluation.

The framework combines raw weather measurements with cyclical calendar features, lag variables, rolling statistics, and difference features. A chronological train-test split and time-series cross-validation are used to preserve the temporal structure of the forecasting problem.

Six primary models are evaluated, including Linear Regression, Ridge, Lasso, Random Forest, Gradient Boosting, and a Multi-Layer Perceptron neural network. Persistence and monthly climatology provide additional baseline references. Model performance is measured using MAE, RMSE, and R^2 .

The feature-engineering ablation experiment provides an additional assessment of whether temporal transformations contribute useful predictive information. The generated diagnostic plots and Random Forest feature importance analysis further support interpretation of the forecasting models.

The final numerical conclusions should be based on the experimental results obtained from the selected weather dataset. Future work can extend the framework to multiple forecast horizons, additional weather targets, more advanced sequence models, and larger meteorological datasets.

Declaration of Competing Interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The availability of the weather dataset depends on the source used for the experimental study. The dataset source and access information should be specified here before final submission.

Acknowledgements

The author would like to acknowledge the academic guidance and support received during the development of this research work.

References

- [1] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, Springer, 2009.
- [2] G. James, D. Witten, T. Hastie, R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.
- [3] L. Breiman, Random Forests, *Machine Learning*, 45(1), 5–32, 2001.
- [4] J. H. Friedman, Greedy Function Approximation: A Gradient Boosting Machine, *The Annals of Statistics*, 29(5), 1189–1232, 2001.

- [5] A. E. Hoerl, R. W. Kennard, Ridge Regression: Biased Estimation for Nonorthogonal Problems, *Technometrics*, 12(1), 55–67, 1970.
- [6] R. Tibshirani, Regression Shrinkage and Selection via the Lasso, *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288, 1996.
- [7] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning Representations by Back-Propagating Errors, *Nature*, 323, 533–536, 1986.
- [8] S. Hochreiter, J. Schmidhuber, Long Short-Term Memory, *Neural Computation*, 9(8), 1735–1780, 1997.
- [9] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed., Wiley, 2015.

Appendix A. Source Code Repository

The complete source code, implementation files, and related resources used for the weather forecasting study are available in the following GitHub repository:

`https://github.com/vrs79066/weather-forecasting`

The repository contains the implementation of the data preprocessing, feature engineering, machine learning models, neural network model, evaluation procedures, and visualization used in this study.